

# MAJOR PROJECT TASK

## DataShield – Segmented Secure Analytics

### Pipeline

(Advanced Linux + AWS Networking Capstone Project)

| Student Details | Information                             |
|-----------------|-----------------------------------------|
| Name            | H BASAVA PRABHU                         |
| Course          | AWS & Linux Administration              |
| Institute       | IT Skill Nest                           |
| Project         | DataShield – Segmented Secure Analytics |

#### Table of Contents

1. Introduction
2. System / Process Overview
3. Implementation / Execution Details
4. Screenshots / Evidence
5. Results / Output
6. Challenges Faced
7. Conclusion

### Abstract

The **DataShield – Segmented Secure Analytics Pipeline** is a secure, cloud-based analytics platform developed using Linux and Amazon Web Services (AWS) to demonstrate secure, multi-tier data ingestion, processing, storage, and retrieval. The platform is designed to securely collect incoming data, forward it internally for analysis, process and store it durably, maintain encrypted backups, and make processed results retrievable through a backend API, while ensuring strict network isolation, automation, and traffic-flow

security. It simulates a real-world segmented enterprise environment where data submitted through a public entry point passes through multiple isolated processing tiers — each governed by dedicated subnets, routing rules, and security boundaries — before its metadata becomes queryable by an authorized backend service.

The project integrates Linux system administration concepts such as automated job scheduling using Cron and disk-level encryption using LUKS (Linux Unified Key Setup), with AWS cloud services including Virtual Private Cloud (VPC), EC2, S3, IAM, RDS (MySQL), Application Load Balancer (ALB), Auto Scaling Group (ASG), AWS Lambda, API Gateway. These technologies work together to create a secure, automated, and network-isolated infrastructure in which each compute tier — data collection, data analysis, backend service, database, and archival storage — resides in its own dedicated subnet with tightly scoped routing and security group rules.

The platform follows a least-privilege security model by implementing dedicated security groups per resource, private-only database access, NAT-based outbound-only internet access for internal tiers, and a fully isolated, LUKS-encrypted archival tier with no internet connectivity. Automated Cron-driven workflows on the Collector and Analyzer instances eliminate manual intervention in the data-forwarding, processing, and backup pipeline, while an S3-triggered Lambda function automatically records processing metadata into a private RDS database. An Auto Scaling Group ensures the backend API tier scales between a minimum of 2 and a maximum of 6 instances based on CPU utilization, and an Application Load Balancer ensures the backend service is reachable only through a controlled, load-balanced entry point. Overall, the DataShield project demonstrates practical implementation of secure VPC networking, routing logic, and Linux-based automation within an AWS cloud environment, addressing real-world challenges related to network isolation, traffic-flow control, automation, and security-boundary enforcement. The project provides hands-on experience in designing and deploying secure, segmented cloud architectures and serves as a practical foundation for careers in Cloud Computing, Linux Administration, DevOps, and Network Security Engineering.

**Keywords:** DataShield, Linux Administration, Amazon Web Services (AWS), Cloud Computing, Secure Data Ingestion, Data Processing, Virtual Private

Cloud (VPC), Amazon EC2, Amazon S3, IAM, Amazon RDS (MySQL), Application Load Balancer (ALB), Auto Scaling Group (ASG), AWS Lambda, API Gateway, Cron Jobs, LUKS Encryption, Network Isolation, Automation, Cloud Security, High Availability, Scalability, Data Encryption.

## **Introduction**

DataShield – Segmented Secure Analytics Pipeline is a cloud-based capstone project developed using Linux and Amazon Web Services (AWS). The project is designed to provide a secure, scalable, and automated platform for collecting, forwarding, processing, storing, and archiving data across a segmented, multi-tier network architecture. It simulates a real-world secured enterprise workflow where data passes through multiple isolated stages, including secure HTTPS ingestion, internal transfer between processing tiers, automated processing and cloud storage, encrypted archival, and database-backed metadata retrieval through a backend API. The project integrates various AWS services such as VPC, EC2, S3, IAM, RDS (MySQL), Application Load Balancer (ALB), Auto Scaling Group (ASG), AWS Lambda, and API Gateway with Linux administration concepts including LUKS disk encryption and Cron job automation. These technologies work together to ensure strict network isolation, automated data movement, and high availability of the backend service.

The primary objective of DataShield is to demonstrate practical knowledge of VPC networking, routing logic, and security-boundary enforcement, while implementing Linux-based automation and encryption within an AWS cloud infrastructure. This project provides hands-on experience in designing and deploying a segmented cloud architecture where each tier — data collection, data analysis, backend service, database, and archival storage — is isolated using dedicated subnets and routing rules, and where traffic is permitted to flow only through explicitly defined paths.

The platform follows industry best practices by implementing least-privilege security group design, private-only database access, outbound-only internet access for internal processing tiers via NAT Gateway, and a fully isolated LUKS-encrypted archival tier with no internet connectivity at all. Sensitive resources such as the database and internal processing instances are deployed in private

subnets with no public IP addresses, while the only public-facing entry points are the Application Load Balancer and API Gateway, both scoped to forward traffic to a single, specific internal destination. Automation through Linux Cron jobs on the Collector and Analyzer instances, combined with an S3-triggered AWS Lambda function, ensures that data is forwarded, processed, uploaded, and recorded in the database efficiently without manual intervention. Amazon S3 provides durable and versioned storage for processed output, while an Auto Scaling Group ensures the backend API tier automatically scales between 2 and 6 instances based on CPU utilization to maintain availability.

Overall, DataShield – Segmented Secure Analytics Pipeline serves as a comprehensive learning project that bridges academic networking concepts with real-world cloud implementation. It provides practical exposure to secure VPC architecture, routing and security-group design, Linux-based automation and encryption, and infrastructure deployment, making it highly relevant for careers in Cloud Computing, AWS Administration, Linux System Administration, DevOps, and Network Security Engineering.

## **Purpose of the Project**

The primary purpose of the DataShield – Segmented Secure Analytics Pipeline project is to design and implement a secure, network-isolated, and automated analytics platform using Linux and Amazon Web Services (AWS). The project simulates a real-world segmented enterprise environment where data is securely ingested, forwarded internally for analysis, processed and stored in the cloud, archived in an encrypted vault, and made retrievable through a private database-backed backend API. This project aims to provide practical, hands-on experience in VPC networking, security-group design, and Linux-based automation by integrating technologies such as VPC, EC2, S3, IAM, RDS (MySQL), Application Load Balancer (ALB), Auto Scaling Group (ASG), AWS Lambda, and API Gateway. It also demonstrates the implementation of Linux features including Cron job automation and LUKS disk encryption.

Overall, the purpose of the DataShield project is to bridge the gap between theoretical networking knowledge and practical implementation by providing real-world experience in designing, deploying, and securing a segmented

cloud infrastructure. It prepares students for careers in Cloud Computing, AWS Administration, Linux System Administration, DevOps, and Network Security Engineering by exposing them to industry-standard tools, technologies, and best practices in network isolation and traffic-flow control.

## Objectives

The main objectives of the DataShield – Segmented Secure Analytics Pipeline are as follows:

1. To design and implement a secure, segmented cloud-based analytics platform using Linux and Amazon Web Services (AWS).
2. To create a secure data ingestion system that accepts data through a secure HTTPS endpoint, routed via API Gateway and a dedicated Application Load Balancer, while keeping the underlying Collector instance fully private with no direct public exposure.
3. To automate the internal data-forwarding and processing workflow using Linux shell scripting and Cron jobs on the Collector and Analyzer instances, eliminating manual intervention and ensuring reliable, scheduled data movement.
4. To implement Linux system administration concepts, including Cron-based job scheduling and LUKS disk encryption for securing archived data at rest.
5. To design a secure AWS network architecture by configuring a Virtual Private Cloud (VPC), public and private subnets, an Internet Gateway, a NAT Gateway, dedicated Route Tables per tier, and least-privilege Security Groups for every resource.
6. To securely store processed content in Amazon S3 with versioning, lifecycle policies, server-side encryption, and fully restricted public access.

## 7. System / Process Overview

The DataShield – Segmented Secure Analytics Pipeline is designed as a secure, automated, and network-isolated cloud-based system for

handling data ingestion, processing, and retrieval using Linux and Amazon Web Services (AWS). The platform follows a structured pipeline in which incoming data is securely ingested, forwarded internally, processed, stored, archived, and made retrievable through a private, database-backed backend service. The architecture combines Linux system administration with AWS cloud infrastructure to ensure network isolation, automation, high availability, and secure traffic flow at every hop.

The system begins when a client — such as the platform's HTML-based ingestion interface — submits data to a public endpoint served by Amazon API Gateway. API Gateway forwards the request to a dedicated, internet-facing Application Load Balancer that fronts the Collector EC2 instance, which itself remains fully private with no direct public IP address. The Collector instance, running a lightweight Python-based HTTPS listener, accepts the incoming data, saves it temporarily to a local directory, and records the event in a log file for traceability.

An automated Linux Cron job on the Collector instance runs every five minutes, checking for newly received files. Any files found are securely transferred via SCP (Secure Copy Protocol) over SSH to the Analyzer EC2 instance, and the local copy on the Collector is removed only after a successful transfer, ensuring no data is lost if a transfer fails. Every forwarding attempt, successful or failed, is logged for auditability.

On the Analyzer instance, a second automated Cron job runs every ten minutes to process any files that have arrived. The processing script performs a basic validation check to reject empty or invalid files, then uploads the validated file to a dedicated Amazon S3 bucket under a date-partitioned prefix, and simultaneously transfers a raw backup copy to the isolated Archive EC2 instance via SCP. The Archive instance stores this backup on a Logical Volume Manager-independent, dedicated EBS volume encrypted using LUKS (Linux Unified Key Setup), ensuring that even direct disk access without the correct passphrase cannot reveal the archived content. The Archive instance has no outbound internet connectivity at all, enforced independently at both the network-routing level and the security-group level.

Whenever a new file is uploaded to the Amazon S3 bucket, an S3 event notification automatically invokes an AWS Lambda function. The Lambda function, running inside the same VPC as a private network

resource, connects securely to a private Amazon RDS MySQL database and inserts a metadata record containing the file name, processing timestamp, and processing status. This database is fully private, with public accessibility disabled at the instance level and network access restricted to only the Lambda function, the Service EC2 tier, and the administrative bastion host.

The Service EC2 tier, running inside an Auto Scaling Group configured for a minimum of two and a maximum of six instances, hosts a backend API that queries the RDS database to retrieve the latest processing records. This tier is reachable only through a dedicated Application Load Balancer, which distributes incoming requests across the healthy Service EC2 instances and continuously performs health checks to ensure availability. The Auto Scaling Group automatically adjusts the number of running instances based on average CPU utilization, maintaining reliability under varying load.

Throughout the entire pipeline, every stage — ingestion, forwarding, processing, backup, and metadata recording — is fully automated through Cron scheduling and AWS-native event triggers, requiring no manual intervention once data enters the system, and every action is logged locally on its respective instance for complete traceability.

## **High-Level Architecture**

The DataShield – Segmented Secure Analytics Pipeline is designed as a secure, segmented, and highly available cloud architecture using Linux and Amazon Web Services (AWS). The architecture follows a multi-tier design in which each compute or data resource occupies its own dedicated subnet, with network access explicitly controlled through routing rules and least-privilege security groups. The platform integrates secure networking, automated data processing, cloud storage, private database management, and controlled API-based retrieval while maintaining strict traffic isolation at every layer.

At the core of the architecture is a Virtual Private Cloud (VPC) with the CIDR block 10.60.0.0/16, providing a single isolated networking environment for the entire platform. The VPC is divided into seven subnets: two Public-Edge subnets (across two Availability Zones) hosting the Application Load Balancers, and five private or isolated subnets — one each for the Data Collector EC2 instance, the Analyzer EC2 instance, the Service EC2 tier, the RDS database, and the Archive EC2 instance. The public subnets communicate with

the internet through an Internet Gateway, private compute subnets reach the internet in an outbound-only capacity through a single NAT Gateway deployed in Public-Edge-Subnet-A, and the database and archive subnets are configured with no internet route at all — neither through the Internet Gateway nor the NAT Gateway.

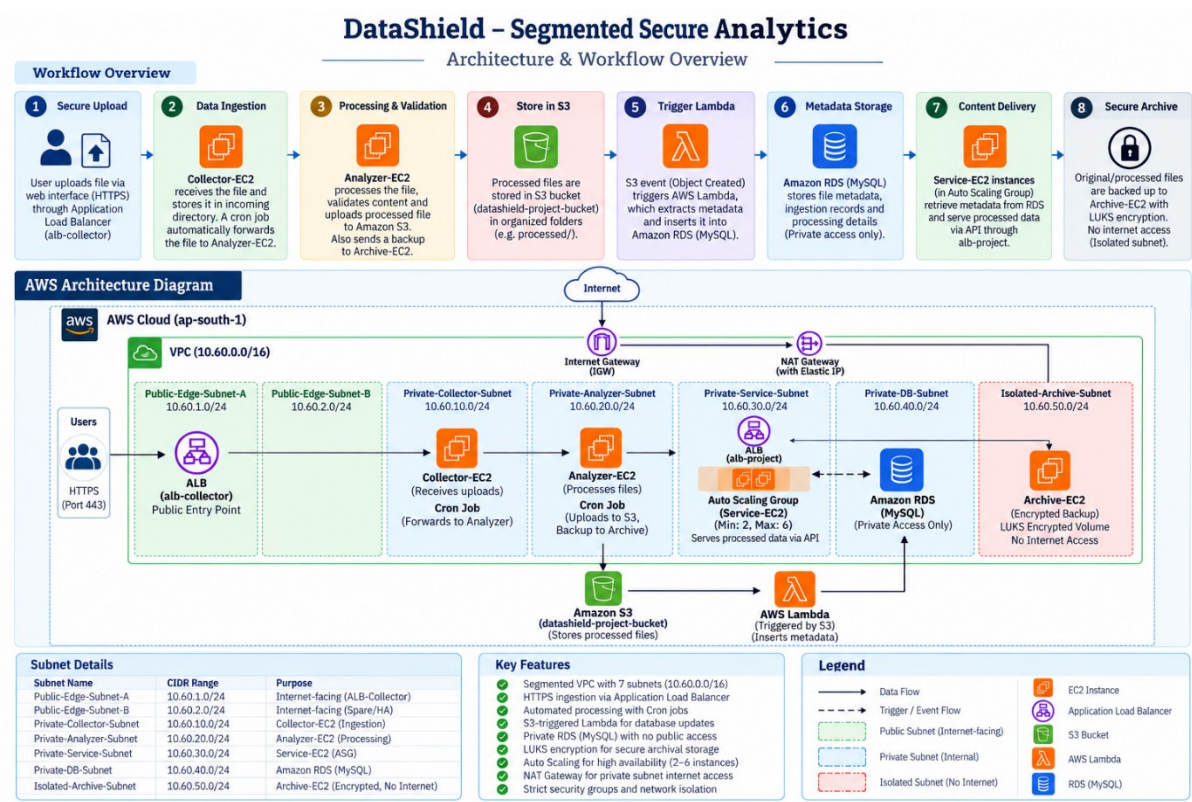
Linux-based EC2 instances deployed across the private subnets handle data ingestion, internal forwarding, processing, backup, and automation. These instances are configured with Linux administration features such as Cron job scheduling and LUKS disk encryption to automate the data pipeline and secure archived data at rest. Incoming API requests destined for the backend service are routed through a dedicated Application Load Balancer, which distributes traffic across the Service EC2 instances. A second, separate Application Load Balancer fronts the Collector EC2 instance specifically to allow Amazon API Gateway to reach it without exposing the Collector instance itself to the public internet. An Auto Scaling Group automatically increases or decreases the number of Service EC2 instances, within a configured range of two to six, based on CPU utilization, ensuring availability and fault tolerance during varying workloads.

Processed data is securely stored in an Amazon S3 bucket with versioning, server-side encryption, lifecycle policies for automatic storage-class transition, and fully blocked public access. When new content is uploaded to S3 under the processed-data prefix, an AWS Lambda function — itself deployed inside the VPC's private subnet — is automatically triggered to record metadata in the Amazon RDS MySQL database. The Amazon RDS MySQL database stores metadata records related to file processing, including file names, processing timestamps, and status. The database is deployed in a fully private subnet, has public accessibility disabled at the instance level, and is reachable only from the Service EC2 tier, the Lambda function, and an administrative bastion host used for management and verification.

Amazon API Gateway exposes a REST API endpoint that accepts incoming data submissions and routes them, via the dedicated Application Load Balancer, to the Collector EC2 instance. Overall, the high-level architecture of DataShield integrates Linux administration with AWS cloud networking services to create a secure, segmented, and automated analytics platform. The architecture ensures secure data ingestion, isolated internal processing, reliable and

versioned storage, automated metadata recording, and controlled API-based data retrieval, while following the principle that traffic may only flow through explicitly defined and justified network paths.

## Workflow / Methodology



The DataShield – Segmented Secure Analytics Pipeline follows a structured and automated workflow for secure data ingestion, collection, analysis, processing, storage, metadata management, and service delivery. The entire workflow is implemented using Linux administration and AWS cloud services while maintaining network segmentation, controlled traffic flow, secure isolation, automation, and scalability.

### **1. Secure Data Upload**

The workflow begins when a user submits data through the application interface. The request is routed through API Gateway to the ingestion path. ALB-Collector acts as the load-balancing entry point for the Collector tier and forwards the request to the Collector EC2 instance located in the Private-Collector-Subnet. The Collector securely receives and temporarily stores the incoming raw data.

### **2. Data Collection**

The Collector EC2 performs the initial data collection operation. Incoming files are stored temporarily before being forwarded to the Analyzer tier. The Collector is placed in a private subnet and does not have direct database connectivity. Its Security Group allows only the communication required for the Collector workflow.

### **3. Automated Data Transfer**

Linux automation is used to transfer collected data from the Collector EC2 to the Analyzer EC2. A scheduled Cron job executes the forwarding script every 5 minutes. The collected files are transferred through the explicitly permitted private communication path (SCP over SSH), ensuring that the movement between the Collector and Analyzer remains controlled and traceable.

### **4. Data Validation and Processing**

The Analyzer EC2, located in the Private-Analyzer-Subnet, receives the collected data and performs validation and processing. The processing workflow is automated using Linux scripts and a Cron job scheduled every 10 minutes. Processing activities, including validation successes and failures, are recorded in logs so that the movement and execution of data can be monitored and traced.

### **5. Processed Data Storage**

After processing is completed, the processed output is uploaded from the

Analyzer EC2 to the Amazon S3 bucket. The S3 bucket is configured with versioning and lifecycle rules to provide controlled storage management. Public access to the bucket is restricted, and the Analyzer uses its dedicated IAM role, scoped only to the PutObject action, to perform the required S3 operations.

## **6. Secure Raw Data Archival**

Raw data is maintained as a backup on the Archive EC2, which is located in the Isolated-Archive-Subnet. The archive storage is protected using LUKS (Linux Unified Key Setup) encryption on a dedicated EBS volume, as required by the project. The Archive subnet has no IGW or NAT route, and the Archive Security Group permits no outbound traffic at all, preventing internet connectivity and providing a defense-in-depth security boundary for archived data.

## **7. S3 Event and Metadata Processing**

When the processed data is uploaded to Amazon S3, the configured S3 event notification triggers the AWS Lambda function. Lambda, deployed within the VPC's private subnet, processes the information received from the S3 event and updates the required metadata in the Amazon RDS MySQL database.

## **8. Private Database Management**

The RDS MySQL database is deployed in the Private-DB-Subnet and remains privately accessible, with public accessibility disabled at the instance level. The RDS Security Group permits MySQL traffic on port 3306 only from three explicitly authorized sources: the Service EC2 Security Group (for metadata retrieval), the Lambda Security Group (for metadata insertion), and the administrative Bastion Security Group (for verification and setup). This ensures the database cannot be accessed from the public internet or from any unauthorized instance.

## **9. Service and Metadata Retrieval**

The Service EC2 instances are deployed in the Private-Service-Subnet and operate behind ALB-Project. When a client requests metadata information, ALB-Project forwards the request to a healthy Service EC2 instance. The Service EC2 communicates with RDS through private VPC connectivity and retrieves the required information.

## **10. Application Load Balancing**

The project uses two Application Load Balancers for separate traffic paths.

ALB-Collector handles the Collector/ingestion path, while ALB-Project handles the Service/application path. Both ALBs use the two public edge subnets, Public-Edge-Subnet-A and Public-Edge-Subnet-B, providing the required public entry points while keeping the backend EC2 instances in private subnets.

### **11. High Availability and Scalability**

The Service EC2 tier operates inside an Auto Scaling Group using a Launch Template. The Auto Scaling Group maintains a minimum of 2 instances and can scale up to a maximum of 6 instances based on CPU utilization. The Application Load Balancer distributes requests to healthy Service EC2 instances, providing availability and scalable service delivery.

### **End-to-End Workflow Summary**

1. User submits data through the application interface.
2. API Gateway receives the ingestion request and routes it to the Collector path.
3. ALB-Collector forwards the request to Collector EC2.
4. Collector EC2 temporarily stores the incoming raw data.
5. Linux Cron automation transfers collected data to Analyzer EC2.
6. Analyzer EC2 validates and processes the collected data.
7. Processed output is uploaded to Amazon S3.
8. Raw data is maintained as a backup on the isolated Archive EC2.
9. S3 triggers Lambda after the configured object event.
10. Lambda updates metadata in the private RDS MySQL database.
11. ALB-Project forwards service requests to healthy Service EC2 instances.
12. Service EC2 retrieves required metadata from RDS through private connectivity.
13. Auto Scaling maintains the Service tier between 2 and 6 instances.

This workflow ensures that data movement is automated, logged, controlled, and traceable, while each tier remains protected by its dedicated subnet, route table, and Security Group.

## Implementation / Execution Details

### AWS Infrastructure Setup

The AWS infrastructure for the DataShield – Segmented Secure Analytics Pipeline was designed and implemented with a strong focus on network security, traffic isolation, controlled connectivity, and scalability. The architecture uses a single custom VPC with multiple dedicated subnets, route tables, Internet Gateway, NAT Gateway, Security Groups, and the AWS services required by the project.

A custom VPC was created with the CIDR block 10.60.0.0/16. Within this VPC, seven dedicated subnets were created according to the role of each component. Public-Edge-Subnet-A (10.60.1.0/24) and Public-Edge-Subnet-B (10.60.2.0/24) are used for the Application Load Balancers. Private-Collector-Subnet (10.60.10.0/24), Private-Analyzer-Subnet (10.60.20.0/24), and Private-Service-Subnet (10.60.30.0/24) contain the application and processing EC2 workloads. Private-DB-Subnet (10.60.40.0/24) is reserved for RDS, while Isolated-Archive-Subnet (10.60.50.0/24) is used for the Archive EC2.

An Internet Gateway (IGW) was attached to the VPC and is used only by the public edge subnets. A NAT Gateway was deployed in Public-Edge-Subnet-A with an allocated Elastic IP. The Collector, Analyzer, and Service private subnets use the NAT Gateway for outbound-only connectivity. Four separate route tables were configured: the Public Route Table routes internet-bound traffic through the IGW; the Private Route Table routes internet-bound traffic through the NAT Gateway with no IGW route; and the Database and Archive Route Tables each contain only local VPC routes, with no internet path of any kind, preventing direct or NAT-based internet connectivity to these restricted resources.

Security Groups were configured as stateful virtual firewalls to control communication between the different tiers, following least-privilege design. The ALB Security Group permits HTTP/HTTPS traffic from the internet and allows outbound communication toward the Service tier only. The Service-EC2 Security Group accepts HTTP traffic only from the ALB and permits the required RDS and S3 communication. The Collector and Analyzer Security Groups restrict communication to their respective application paths and

explicitly exclude any rule permitting Collector-to-RDS communication. The RDS Security Group permits MySQL traffic on port 3306 only from the Service tier, the Lambda function, and the administrative Bastion host, with no outbound rules at all. The Archive Security Group allows SSH only from the approved administrator IP and the Bastion host, and permits no outbound traffic whatsoever.

Overall, the AWS infrastructure establishes the required foundation for DataShield by combining VPC segmentation, controlled routing, NAT-based private connectivity, least-privilege Security Groups, and restricted database and archive networks. This ensures that communication occurs only through explicitly permitted paths as defined by the project architecture.

---

## **EC2 and Linux Configuration**

After establishing the AWS network infrastructure, the required Linux EC2 instances were deployed in their respective subnets according to their roles in the DataShield pipeline. The Collector, Analyzer, Service, and Archive EC2 workloads are separated into dedicated network segments so that each system performs only its assigned function.

The Collector EC2, located in the Private-Collector-Subnet, runs a lightweight Python-based HTTPS listener responsible for receiving incoming data and temporarily storing the collected raw data before forwarding it to the Analyzer EC2. The Collector does not have direct database connectivity, maintaining separation between the ingestion and database tiers, enforced at the security-group level by the deliberate absence of any Collector-to-RDS rule.

The Analyzer EC2, located in the Private-Analyzer-Subnet, performs validation and processing of the collected data using a Linux shell script scheduled via Cron. After processing, the output is uploaded to Amazon S3 using the AWS CLI, while a raw backup copy is transferred to the Archive EC2 via SCP.

The Service EC2 tier is deployed in the Private-Service-Subnet and hosts a Python Flask backend API, served through Gunicorn, that communicates with the private RDS MySQL database. It is accessed only through the Application Load Balancer rather than being directly exposed to external users, and operates as part of an Auto Scaling Group.

The Archive EC2, located in the Isolated-Archive-Subnet, maintains raw data backups on a dedicated, LUKS-encrypted EBS volume. The subnet has no IGW or NAT route, and the instance's Security Group permits no outbound traffic, preventing internet connectivity at two independent layers.

A structured Linux directory arrangement was created on each EC2 system for storing incoming data, processing files, output data, scripts, and logs. Logs are generated throughout the pipeline so that data movement and processing activities remain automated, logged, and traceable.

Overall, the EC2 and Linux configuration establishes separate environments for data collection, analysis, service delivery, and archival storage, while maintaining the network and security boundaries defined in the DataShield architecture.

.

---

## **Automation and Scheduling**

Automation plays an important role in the DataShield project by reducing manual intervention and ensuring that data moves through the processing pipeline consistently. Linux Cron jobs were configured on the Collector and Analyzer EC2 instances to automate file transfer and processing activities.

On the Collector EC2, a Cron job scheduled every 5 minutes executes the forwarding script, which transfers collected files to the Analyzer EC2 through the permitted internal SCP communication path and removes the local copy only after a successful transfer.

On the Analyzer EC2, a Cron job scheduled every 10 minutes executes the processing script, which validates the collected data, uploads the processed output to Amazon S3, transfers a raw backup to the Archive EC2, and records execution information in the appropriate log files.

The execution of the automated tasks was verified through Linux logs and live testing, in which a data submission was traced end-to-end from the Collector's ingestion log, through the Analyzer's processing log, to a confirmed metadata record in the RDS database — with no manual script execution at any stage after the initial submission.

Overall, the use of Cron-based Linux automation, controlled file permissions, scripts, and logging ensures that the DataShield pipeline operates in an automated, reliable, and traceable manner.

---

## **Load Balancer and Traffic Flow**

To provide controlled application access and scalability, two Application Load Balancers were configured as part of the DataShield architecture. Both ALBs use the two public edge subnets, Public-Edge-Subnet-A and Public-Edge-Subnet-B, while the backend EC2 workloads remain in private subnets.

The first load balancer, ALB-Collector, is used for the Collector/ingestion path. It receives ingestion traffic — forwarded from API Gateway via an HTTP proxy integration — and routes requests toward the Collector EC2 in the Private-Collector-Subnet. This ALB was specifically introduced to allow API Gateway to reach the private Collector instance without requiring a Network Load Balancer, which falls outside the project's approved service list; using an Application Load Balancer with a plain HTTP integration avoided this dependency entirely.

The second load balancer, ALB-Project, is used for the Service/application path. It provides the application entry point for the Service tier and forwards requests to the Service EC2 instances located in the Private-Service-Subnet.

Target Groups were configured for the respective backend workloads, using health checks against a dedicated /health endpoint on each backend to determine target availability. Only healthy targets receive traffic.

Security Groups provide an additional layer of traffic control. The ALB Security Groups permit HTTP/HTTPS traffic from the internet, while the Service Security Group permits HTTP traffic only from its ALB, and the Collector Security Group permits traffic on its ingestion port only from its own dedicated ALB's Security Group. This prevents direct unrestricted access to either backend tier.

The Service tier is integrated with an Auto Scaling Group, which maintains the required number of Service EC2 instances between a minimum of 2 and a maximum of 6, scaling based on CPU utilization. As a result, the ALB can

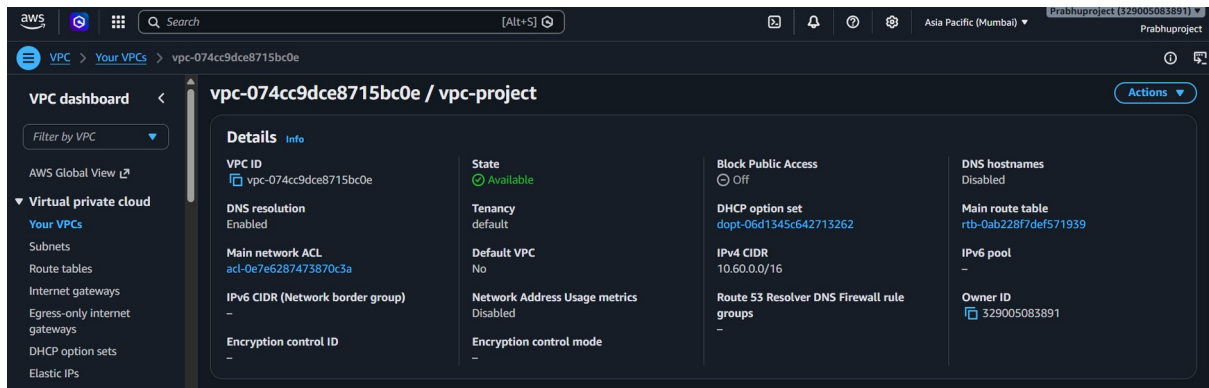
distribute application traffic across the available healthy Service instances while the Auto Scaling Group manages the required capacity.

Overall, the two-ALB architecture separates the Collector ingestion path from the Service application path, while private subnet placement, Security Groups, Target Groups, and Auto Scaling provide controlled traffic flow, isolation, availability, and scalability.

## **Screenshots / Evidence :**

### **VPC**

- VPC CIDR: 10.60.0.0/16
- Single VPC with multiple isolated subnets



## Mandatory Subnets

Public-Edge-Subnet-A 10.60.1.0/24

Public-Edge-Subnet-B 10.60.2.0/24

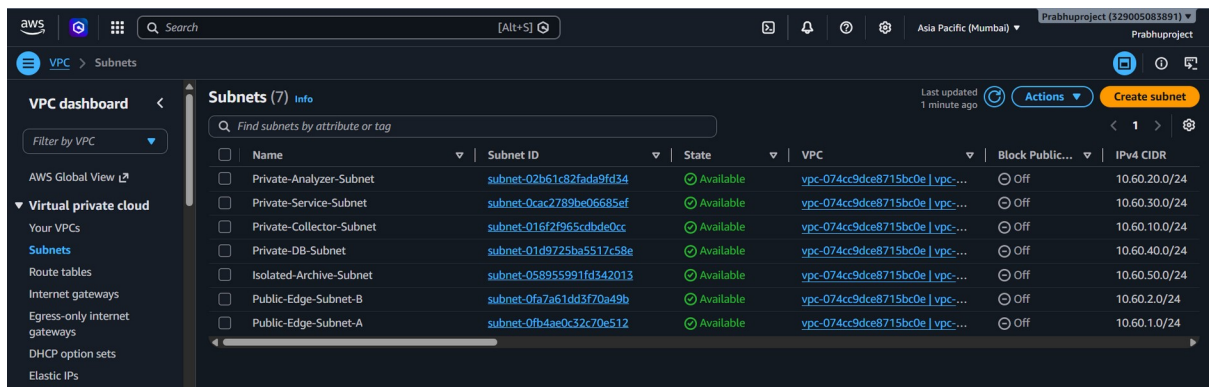
Private-Collector-Subnet 10.60.10.0/24

Private-Analyzer-Subnet 10.60.20.0/24

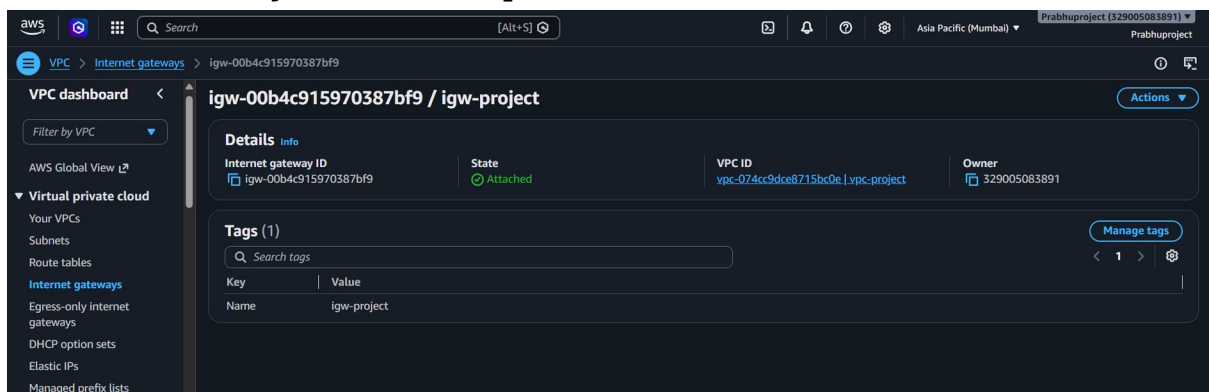
Private-Service-Subnet 10.60.30.0/24

Private-DB-Subnet 10.60.40.0/24

Isolated-Archive-Subnet 10.60.50.0/24

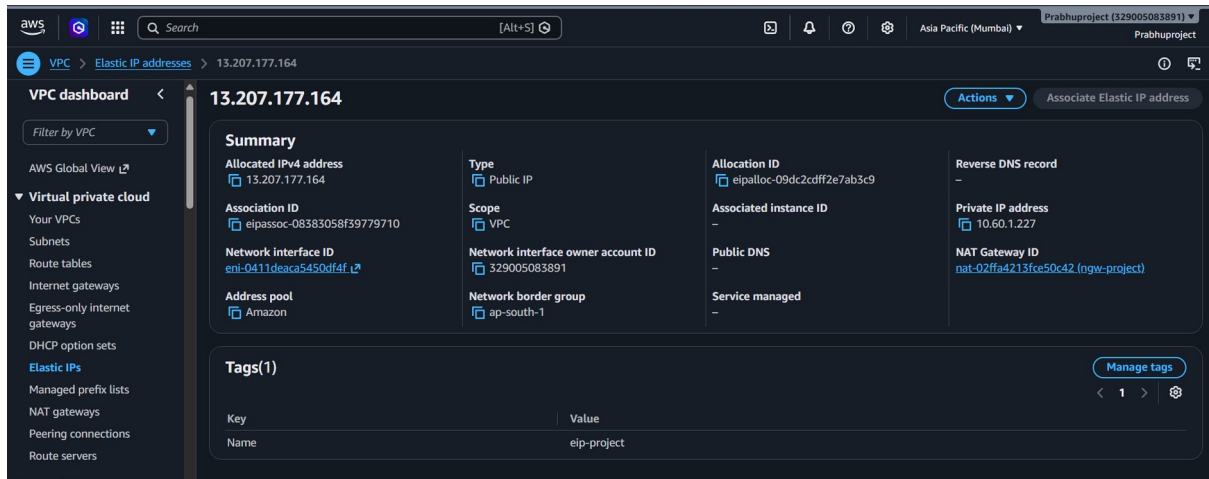


## Internet Gateway (IGW) (with vpc attached)



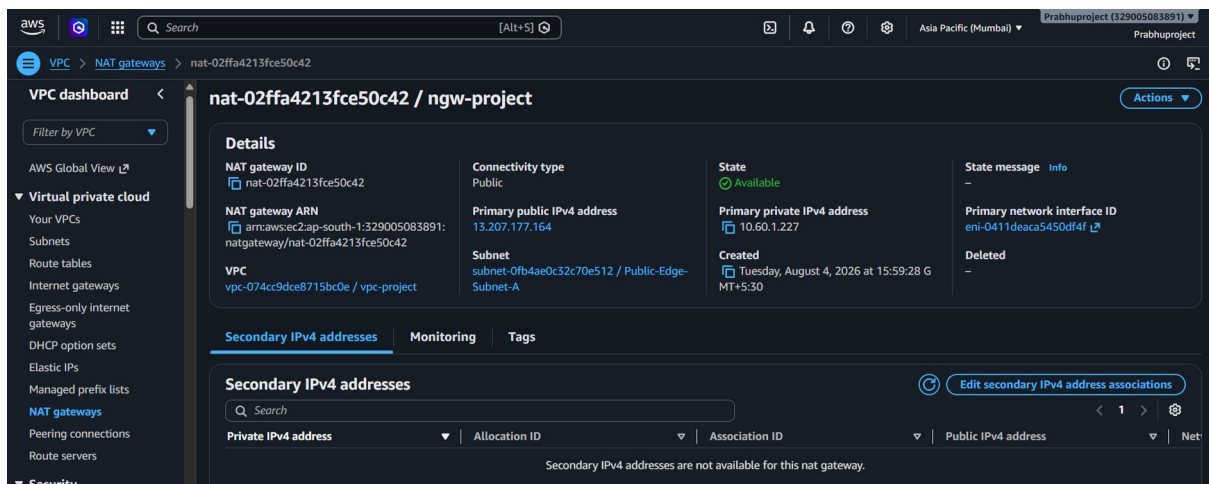
**Internet Gateway (IGW):** The Internet Gateway was attached to the DataShield VPC to provide internet connectivity for the public-edge subnets.

## Elastic ip:



Elastic IP: An Elastic IP address was allocated and associated with the NAT Gateway to provide a stable public IP for outbound internet access from the private subnets.

## NAT gateway:



**NAT Gateway:** The NAT Gateway was created in the Public-Edge-Subnet-A and associated with an Elastic IP address. It provides outbound internet connectivity for the private Collector, Analyzer, and Service subnets while keeping these resources from receiving direct inbound internet access.

## Route Tables:

**Route Tables:** Four route tables were configured to control traffic within the DataShield VPC: Public, Private, Database, and Archive route tables. The Public route table uses the Internet Gateway for internet access, while the Private route table routes outbound traffic through the NAT Gateway for the private Collector, Analyzer, and Service subnets. The Database and Archive route tables maintain isolated local routing for the RDS and Archive subnets, supporting the required network segmentation.

## Public Route Table

## rtb-028f381c6ed68de7c / Public Route Table

[Actions](#)

### Details [Info](#)

Route table ID  
 rtb-028f381c6ed68de7c

Main  
 No

Explicit subnet associations  
[2 subnets](#)

Edge associations  
-

VPC  
[vpc-074cc9dce8715bc0e](#) | [vpc-project](#)

Owner ID  
 329005083891

[Routes](#)[Subnet associations](#)[Edge associations](#)[Route propagation](#)[Tags](#)[Related resources](#)

### Routes (2)

[Both](#)[Edit routes](#)

| Destination  | Target                                | Status | Propagated | Route Origin       |
|--------------|---------------------------------------|--------|------------|--------------------|
| 0.0.0.0/0    | <a href="#">igw-00b4c915970387bf9</a> | Active | No         | Create Route       |
| 10.60.0.0/16 | local                                 | Active | No         | Create Route Table |

## Private Route Table (Collector / Analyzer / Service):

## rtb-021ea0db5ad3409a6 / Private Route Table

[Actions](#)

### Details [Info](#)

Route table ID  
 rtb-021ea0db5ad3409a6

Main  
 No

Explicit subnet associations  
[3 subnets](#)

Edge associations  
-

VPC  
[vpc-074cc9dce8715bc0e](#) | [vpc-project](#)

Owner ID  
 329005083891

[Routes](#)[Subnet associations](#)[Edge associations](#)[Route propagation](#)[Tags](#)[Related resources](#)

### Routes (2)

[Both](#)[Edit routes](#)

| Destination  | Target                                | Status | Propagated | Route Origin       |
|--------------|---------------------------------------|--------|------------|--------------------|
| 0.0.0.0/0    | <a href="#">nat-02ffa4213fce50c42</a> | Active | No         | Create Route       |
| 10.60.0.0/16 | local                                 | Active | No         | Create Route Table |

## Database Route Table:

## rtb-0985dd673fb76a1cc / Database Route Table

[Actions](#)

### Details [Info](#)

Route table ID  
 rtb-0985dd673fb76a1cc

Main  
 No

Explicit subnet associations  
[subnet-01d9725ba5517c58e](#) / [Private-DB-Subnet](#)

Edge associations  
-

VPC  
[vpc-074cc9dce8715bc0e](#) | [vpc-project](#)

Owner ID  
 329005083891

[Routes](#)[Subnet associations](#)[Edge associations](#)[Route propagation](#)[Tags](#)[Related resources](#)

### Routes (1)

[Both](#)[Edit routes](#)

| Destination  | Target | Status | Propagated | Route Origin       |
|--------------|--------|--------|------------|--------------------|
| 10.60.0.0/16 | local  | Active | No         | Create Route Table |

## Archive Route Table:

**rtb-01b426d460b498245 / Archive Route Table** Actions

**Details** Info

**Route table ID**  
rtb-01b426d460b498245

**VPC**  
vpc-074cc9dce8715bc0e | vpc-project

**Main**  
No

**Owner ID**  
329005083891

**Explicit subnet associations**  
subnet-058955991fd342013 / Isolated-Archive-Subnet

**Edge associations**  
-

Routes

Subnet associations

Edge associations

Route propagation

Tags

Related resources

**Routes (1)**

Both Edit routes

| Destination  | Target | Status | Propagated | Route Origin       |
|--------------|--------|--------|------------|--------------------|
| 10.60.0.0/16 | local  | Active | No         | Create Route Table |

## Security groups:

**Security Groups (10)** Info Actions Export security groups to CSV Create security group

| Security group ID    | Security group name | VPC ID                | Description                  |
|----------------------|---------------------|-----------------------|------------------------------|
| sg-07c32461bf6a2823c | alb-sg-collector    | vpc-074cc9dce8715bc0e | alb-sg-collector             |
| sg-05f5d3acf3d1d44c2 | analyzer-sg         | vpc-074cc9dce8715bc0e | Analyzer-EC2 security group  |
| sg-0ce73c6a9b44a409d | SG-Lambda           | vpc-074cc9dce8715bc0e | SG-Lambda                    |
| sg-04f49d3af4509cb9c | rds-sg              | vpc-074cc9dce8715bc0e | RDS security group           |
| sg-026dcde8b24d686e1 | sgbastion           | vpc-074cc9dce8715bc0e | bastion                      |
| sg-003b3106ef671f8f3 | archive-sg          | vpc-074cc9dce8715bc0e | Archive-EC2 security group   |
| sg-0197d59143f71394b | default             | vpc-074cc9dce8715bc0e | default VPC security group   |
| sg-0a3cf40730a96186d | collector-sg        | vpc-074cc9dce8715bc0e | Collector-EC2 security group |
| sg-02908e542125d47e8 | service-sg          | vpc-074cc9dce8715bc0e | Service-EC2 security group   |
| sg-0b8991132dee793f4 | alb-sg              | vpc-074cc9dce8715bc0e | ALB security group           |

## Policy and roles:

### Analyzer Policy and role :

Analyzer Policy and Role: An IAM policy and role were created specifically for the Analyzer EC2 instance to provide controlled access to the required AWS resources. The policy grants the Analyzer the necessary permissions to upload processed files to the DataShield S3 bucket and perform the required operations without providing unnecessary AWS permissions.

analyzer-policy

Info

EditDelete

Policy details

Type

Customer managed

Creation time

August 04, 2026, 17:24 (UTC+05:30)

Edited time

August 04, 2026, 17:41 (UTC+05:30)

ARN

arn:aws:iam::329005083891:policy/analyzer-policy

Permissions

Entities attached

Tags

Policy versions (2)

Last Accessed

Permissions defined in this policy

Info

EditSummaryJSON

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

Search

Allow (1 of 473 services)

Show remaining 472 services

| Service | Access level   | Resource                                                                               | Request condition |
|---------|----------------|----------------------------------------------------------------------------------------|-------------------|
| S3      | Limited: Write | BucketName  string like<br> datashield-project-bucket,<br>ObjectPath  string like  All | None              |

analyzer-role

Info

Delete

Allows EC2 instances to call AWS services on your behalf.

Summary

Edit

Creation date

August 04, 2026, 17:25 (UTC+05:30)

ARN

arn:aws:iam::329005083891:role/analyzer-role

Instance profile ARN

arn:aws:iam::329005083891:instance-profile/analyzer-role

Last activity

7 hours ago

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (1)

Info

SimulateRemoveAdd permissions

You can attach up to 10 managed policies.

Filter by Type

All types

Search

| Policy name     | Type             | Attached entities |
|-----------------|------------------|-------------------|
| analyzer-policy | Customer managed | 1                 |

## Lambda Policy and role :

Lambda Policy and Role: An IAM policy and role were created specifically for the Lambda function to provide the permissions required for its operations. The role allows Lambda to access the necessary S3 and RDS-related resources, enabling it to process S3 event notifications and update the metadata stored in the private RDS database with controlled permissions.

lambda-role

Info

Delete

Allows Lambda functions to call AWS services on your behalf.

Summary

Edit

Creation date

August 04, 2026, 17:28 (UTC+05:30)

ARN

arn:aws:iam::329005083891:role/lambda-role

Last activity

8 hours ago

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (1)

Info

Simulate

Remove

Add permissions

You can attach up to 10 managed policies.

Search

Filter by Type

All types

< 1 >

☐

Policy name

▲

Type

▼

Attached entities

▼

☐

lambda-policy

Customer managed

1

## Application load balancer:

Application Load Balancer: Two Application Load Balancers were configured to distribute traffic to the required backend services through their respective target groups. ALB-Collector handles the ingestion path to the Collector EC2, while ALB-Project routes application requests to the Service EC2 instances, providing controlled access and reliable traffic distribution.

alb-project

Actions

▼ Details

Load balancer type

Application

Status

Active

VPC

vpc-074cc9dce8715bc0e

Load balancer IP address type

IPv4

Scheme

Internet-facing

Hosted zone

ZP97RAFLXTNZK

Availability Zones

subnet-0fa7a61dd3f70a49b ap-south-1a (aps1-az1)  
subnet-0fb4ae0c32c70e512 ap-south-1b (aps1-az3)

Date created

August 5, 2026, 12:21 (UTC+05:30)

Load balancer ARN

arn:aws:elasticloadbalancing:ap-south-1:329005083891:loadbalancer/app/alb-project/3c0129b0b942c865

DNS name

alb-project-1913958840.ap-south-1.elb.amazonaws.com (A Record)

Listeners and rules

Network mapping

Resource map

Security

Monitoring

Integrations

Attributes

Capacity

Tags

Security groups (1)

Edit

A security group is a set of firewall rules that control the traffic to your load balancer.

Security Group ID

Name

Description

Inbound rules count

Outbound rules count

sg-0b8991132dee793f4

alb-sg

ALB security gr...

2 permission entries

1 permission entry



TG-Service

Actions

Details

aws:elasticloadbalancing:ap-south-1:329005083891:targetgroup/TG-Service/064aef7e1e324fa5

Target type

Instance

IP address type

IPv4

Protocol : Port

HTTP: 80

Load balancer

alb-project

Protocol version

HTTP1

VPC

vpc-074cc9dce8715bc0e

2

Total targets

2 Healthy

0 Anomalous

0 Unhealthy

0 Unused

0 Initial

0 Draining

Distribution of targets by Availability Zone (AZ)

Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets

Monitoring

Health checks

Attributes

Tags

Registered targets (2)

Anomaly mitigation: Not applicable

De-register

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

1

2

Settings

|                          | Instance ID         | Name | Port | Zone              | Health status | Health status details | Admini...                          | Overri...                          | Launch...     | Anomaly detectio... |
|--------------------------|---------------------|------|------|-------------------|---------------|-----------------------|------------------------------------|------------------------------------|---------------|---------------------|
| <input type="checkbox"/> | i-0b8944b8b41b01588 | -    | 80   | ap-south-1b (a... | Healthy       | -                     | <input type="radio"/> No override. | <input type="radio"/> No overri... | August 7, ... | Normal              |
| <input type="checkbox"/> | i-08bf67fcd6ed76a76 | -    | 80   | ap-south-1b (a... | Healthy       | -                     | <input type="radio"/> No override. | <input type="radio"/> No overri... | August 7, ... | Normal              |

Launch Template: A Launch Template was created to define the configuration required for the Service EC2 instances launched by the Auto Scaling Group. It contains the required instance settings, security group, IAM role, and user-data script to automatically install and configure the DataShield Service application when a new instance is launched.

launchtemplate-project (lt-03be171eaf3863cc0)

ActionsDelete template

Launch template details

Launch template ID

lt-03be171eaf3863cc0

Launch template name

launchtemplate-project

Default version

1

Owner

arn:aws:iam::329005083891:root

Details

Versions

Template tags

Launch template version details

ActionsDelete template version

Version

1 (Default)

Description

-

Date created

2026-08-05T06:53:55.000Z

Created by

arn:aws:iam::329005083891:root

Instance details

Storage

Resource tags

Network interfaces

Advanced details

AMI ID

ami-035827357e3c7e810

Instance type

t3.micro

Availability Zone

-

Availability Zone Id

-

Key pair name

datashieldproject

Security groups

-

Security group IDs

sg-02908e542125d47e8

## Auto Scaling group:

Auto Scaling Group (ASG): An Auto Scaling Group was configured to manage the Service EC2 instances using the created Launch Template. The ASG was configured with a minimum of 2 instances, desired capacity of 2, and maximum of 6 instances, with CPU utilization used as the scaling condition to automatically adjust the number of Service instances based on workload.

asg-project Capacity overview

Edit

arn:aws:autoscaling:ap-south-1:329005083891:autoScalingGroup:0d607c5b-6200-4eeb-a0bb-ad6264baae1c:autoScalingGroupName/asg-project

Desired capacity

2

Scaling limits

2 - 6

Desired capacity type

Units (number of instances)

Status

At desired capacity

Date created

Wed Aug 05 2026 12:29:07 GMT+0530 (India Standard Time)

Details

Integrations

Automatic scaling

Instance management

Instance refresh

Activity

Monitoring

Tags

Launch template

Edit

Launch template

lt-03be171eaf3863cc0  
launchtemplate-project

AMI ID

ami-035827357e3c7e810

Instance type

t3.micro

Owner

arn:aws:iam::329005083891:root

Version

Default

Security groups

-

Security group IDs

sg-02908e542125d47e8

Create time

Wed Aug 05 2026 12:23:55 GMT+0530 (India Standard Time)

Description

-

Storage (volumes)

-

Key pair name

datashieldproject

Request Spot Instances

No

View details in the launch template console

Network

Edit

Availability Zones

aps1-az3 (ap-south-1b)

Subnet ID

subnet-0cac2789be06685ef

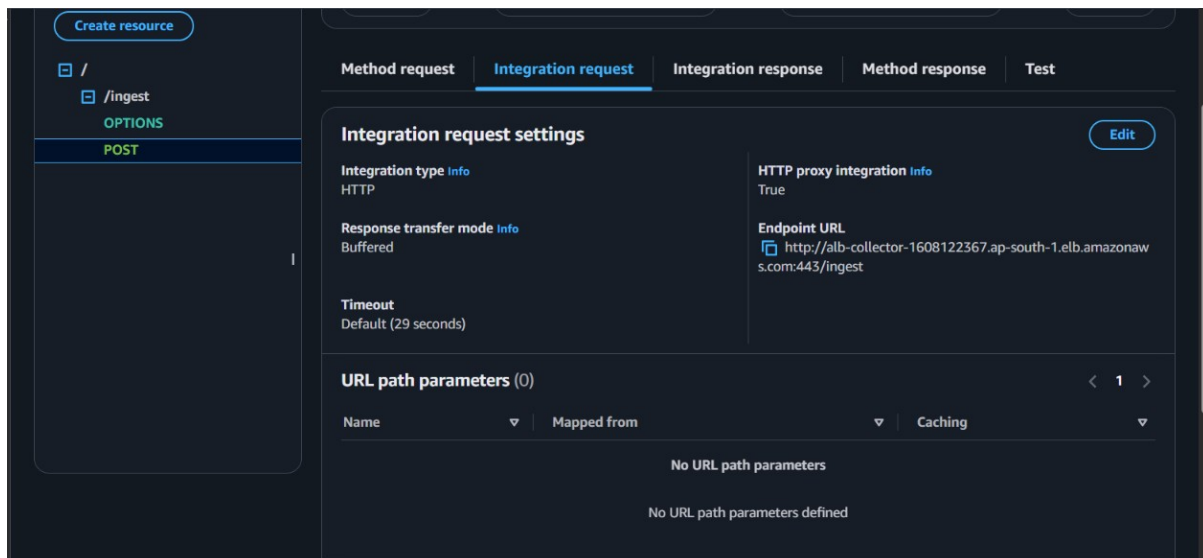
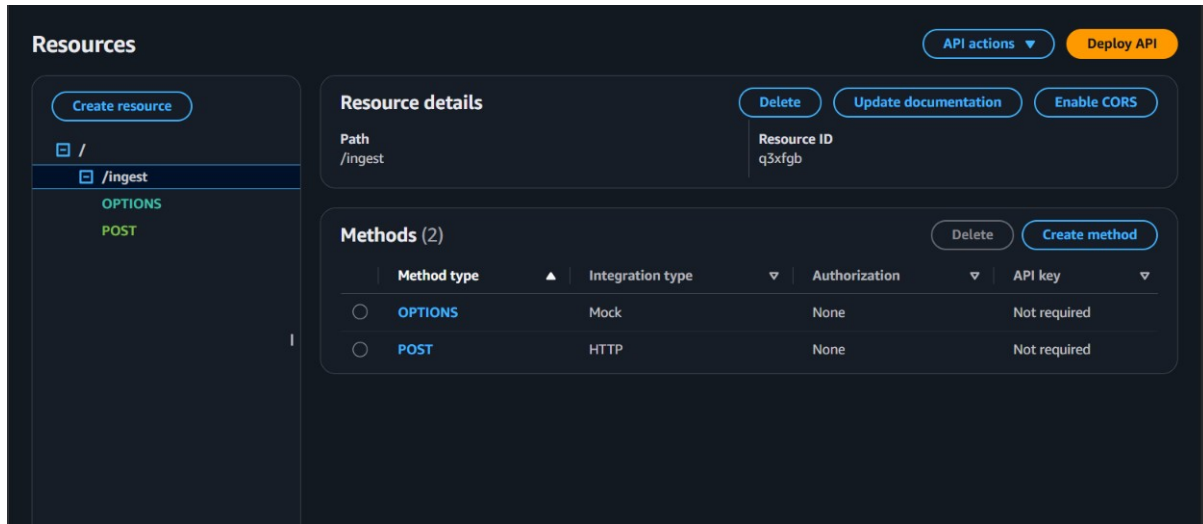
Availability Zone distribution

Balanced best effort

## Api gateway:

API Gateway: An Amazon API Gateway REST API was configured with an /ingest POST endpoint to receive file ingestion requests from the frontend. The POST method uses an HTTP proxy integration with ALB-Collector, forwarding

incoming requests to the Collector EC2 and initiating the DataShield ingestion pipeline.



Stages

prod

Stage actions

Create stage

Stage details

Stage name

prod

Cache cluster

Inactive

Client certificate type

-

Rate

10000

Burst

5000

Default method-level caching

Inactive

Web ACL

-

Client certificate

-

Invoke URL

https://qfbultahm6.execute-api.ap-south-1.amazonaws.com/prod

Active deployment

zuf0yl on September 15, 2026, 15:43 (UTC+05:30)

Logs and tracing

Logging level

Inactive

Detailed metrics

Inactive

Data tracing

Inactive

## Linux Configuration:

### Collector-ec2

Collector EC2: The Collector EC2 instance acts as the initial processing point for incoming files in the DataShield pipeline. It receives ingestion requests through API Gateway and ALB-Collector, stores the incoming files in the designated directory, and uses a scheduled forward.sh script to securely transfer the files to the Analyzer EC2 for further processing.

```

├── collector-key.pem
├── datashield
│   ├── incoming
│   ├── logs
│   │   ├── collector-forward.log
│   │   └── collector-ingest.log
│   └── scripts
│       ├── forward.sh
│       └── ingest_server.py
4 directories, 5 files
[ec2-user@ip-10-60-10-242 ~]$ cat /home/ec2-user/datashield/logs/collector-ingest.log
2026-08-07 12:03:41.878180 Received file: /home/ec2-user/datashield/incoming/data_20260807120341.txt (15 bytes)
2026-08-11 03:06:35.474590 Received file: /home/ec2-user/datashield/incoming/data_20260811030635.txt (15 bytes)
2026-08-11 03:09:45.036202 Received file: /home/ec2-user/datashield/incoming/data_20260811030945.txt (15 bytes)
2026-08-11 03:30:48.402838 Received file: /home/ec2-user/datashield/incoming/data_20260811033048.txt (842 bytes)
2026-08-11 03:32:15.531598 Received file: /home/ec2-user/datashield/incoming/data_20260811033215.txt (842 bytes)
2026-08-11 03:34:44.766253 Received file: /home/ec2-user/datashield/incoming/data_20260811033444.txt (842 bytes)
2026-08-11 03:34:46.911178 Received file: /home/ec2-user/datashield/incoming/data_20260811033446.txt (842 bytes)
2026-08-11 03:34:55.041057 Received file: /home/ec2-user/datashield/incoming/data_20260811033455.txt (842 bytes)
2026-08-11 03:44:02.498716 Received file: /home/ec2-user/datashield/incoming/data_20260811034402.txt (40 bytes)
2026-08-11 13:32:14.677642 Received file: /home/ec2-user/datashield/incoming/data_20260811133214.txt (40 bytes)
2026-08-11 21:26:52.230525 Received file: /home/ec2-user/datashield/incoming/data_20260811212652.txt (842 bytes)
2026-09-15 15:47:42.422316 Received file: /home/ec2-user/datashield/incoming/data_20260915154742.txt (25 bytes)
2026-09-15 16:02:52.306008 Received file: /home/ec2-user/datashield/incoming/data_20260915160252.txt (842 bytes)
2026-09-16 17:35:03.752715 Received file: /home/ec2-user/datashield/incoming/data_20260916173503.txt (241 bytes)
2026-09-18 15:29:56.230464 Received file: /home/ec2-user/datashield/incoming/data_20260918152956.txt (216597 bytes)
[ec2-user@ip-10-60-10-242 ~]$

```

```
[ec2-user@ip-10-60-10-242 ~]$ crontab -l
*/5 * * * * /home/ec2-user/datashield/scripts/forward.sh
[ec2-user@ip-10-60-10-242 ~]$
```

## Analyzer-ec2:

Analyzer EC2: The Analyzer EC2 instance receives files from the Collector EC2 and performs the next stage of the DataShield processing pipeline. A scheduled process.sh script validates and processes the files, uploads the processed data to the S3 bucket, and securely creates a raw backup in the Archive EC2 for storage.

```
[ec2-user@ip-10-60-20-96 ~]$ tree .
.
├── collector-key.pem
├── datashield
│   ├── logs
│   │   └── analyzer-process.log
│   ├── processed
│   │   └── data_20260915160252.txt
│   ├── processing
│   ├── scripts
│   │   └── process.sh
└── 5 directories, 4 files
```

```
[ec2-user@ip-10-60-20-96 ~]$ crontab -l
*/10 * * * * /home/ec2-user/datashield/scripts/process.sh
[ec2-user@ip-10-60-20-96 ~]$ cat /home/ec2-user/datashield/logs/analyzer-process.log
2026-08-11 13:24:38 FAILED Archive backup for data_20260811033215.txt
2026-08-11 13:40:02 Uploaded data_20260811133214.txt to S3
2026-08-11 13:40:02 Backed up data_20260811133214.txt to Archive
2026-08-12 12:20:04 Uploaded data_20260811212652.txt to S3
2026-08-12 12:20:04 Backed up data_20260811212652.txt to Archive
2026-09-15 16:10:03 Uploaded data_20260915160252.txt to S3
2026-09-15 16:10:04 Backed up data_20260915160252.txt to Archive
[ec2-user@ip-10-60-20-96 ~]$ ls /home/ec2-user/datashield/processed
data_20260915160252.txt
[ec2-user@ip-10-60-20-96 ~]$ aws s3 ls s3://datashield-project-bucket/processed/
```

## S3 bucket(versioning + lifecycle rules):

S3 Bucket (Versioning + Lifecycle Rules): The DataShield S3 bucket was configured to store the processed files generated by the Analyzer EC2. Versioning was enabled to maintain multiple versions of objects, while a lifecycle rule was configured to manage objects according to the project's storage requirements, supporting organized and controlled data storage.

datashield-project-bucket

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

Access Points

Bucket overview

AWS Region

Asia Pacific (Mumbai) ap-south-1

Amazon Resource Name (ARN)

arn:aws:s3:::datashield-project-bucket

Creation date

August 4, 2026, 17:39:08 (UTC+05:30)

Bucket Versioning

Edit

Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. [Learn more](#)

Bucket Versioning

Enabled

Multi-factor authentication (MFA) delete

An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To modify MFA delete settings, use the AWS CLI, AWS SDK, or the Amazon S3 REST API. [Learn more](#)

Disabled

datsshield-rule

Info

EditDeleteActions

Lifecycle rule configuration

Lifecycle rule name  
datsshield-rule

Status  
Enabled

Scope  
Filtered

Prefix  
processed/

Object tags  
-

Minimum object size  
-

When no minimum object size is specified, the minimum object size for transitions is determined by the lifecycle configuration. [Learn more](#)

Maximum object size  
-

Review transition and expiration actions

Current version actions

Day 0

- Objects uploaded

↓

Day 30

- Objects move to Standard-IA

↓

Day 180

- Objects expire

Noncurrent versions actions

Day 0

No actions defined.

datashield-project-bucket

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems

Access Points

Objects (1)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

Show versions

1

|  | Name       | Type   | Last modified | Size | Storage class |
|--|------------|--------|---------------|------|---------------|
|  | processed/ | Folder | -             | -    | -             |

2026-09-18/ Copy S3 URI

Objects
Properties

Objects (4)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Show versions
< 1 >
⚙

| <input type="checkbox"/> | Name                                    | Type | Last modified                            | Size     | Storage class |
|--------------------------|-----------------------------------------|------|------------------------------------------|----------|---------------|
| <input type="checkbox"/> | <a href="#">data_20260916173503.txt</a> | txt  | September 18, 2026, 15:10:04 (UTC+05:30) | 241.0 B  | Standard      |
| <input type="checkbox"/> | <a href="#">data_20260918152956.txt</a> | txt  | September 18, 2026, 15:40:04 (UTC+05:30) | 211.5 KB | Standard      |
| <input type="checkbox"/> | <a href="#">data_20260918155514.txt</a> | txt  | September 18, 2026, 16:10:03 (UTC+05:30) | 223.1 KB | Standard      |
| <input type="checkbox"/> | <a href="#">data_20260918161719.txt</a> | txt  | September 18, 2026, 16:30:04 (UTC+05:30) | 113.2 KB | Standard      |

## Archive ec2:

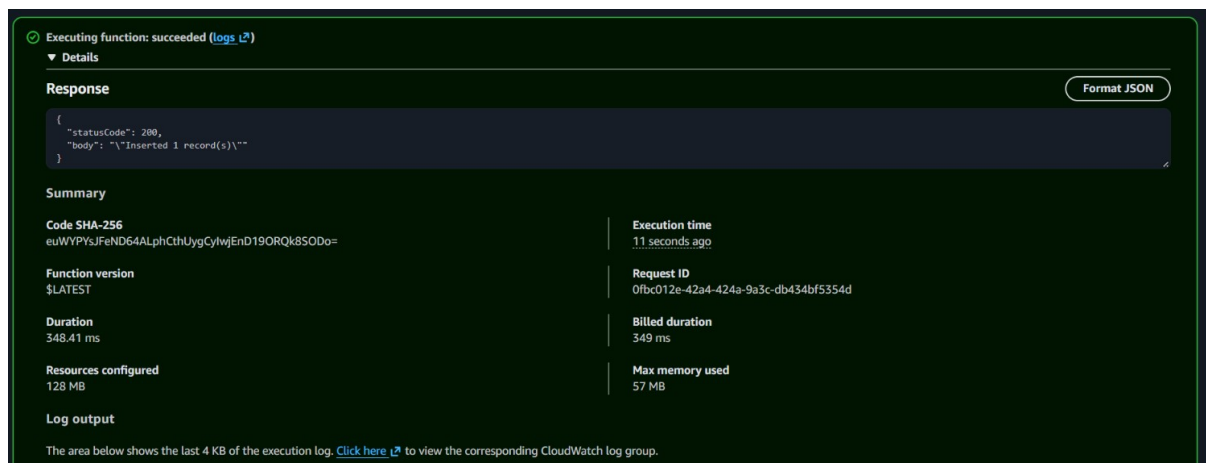
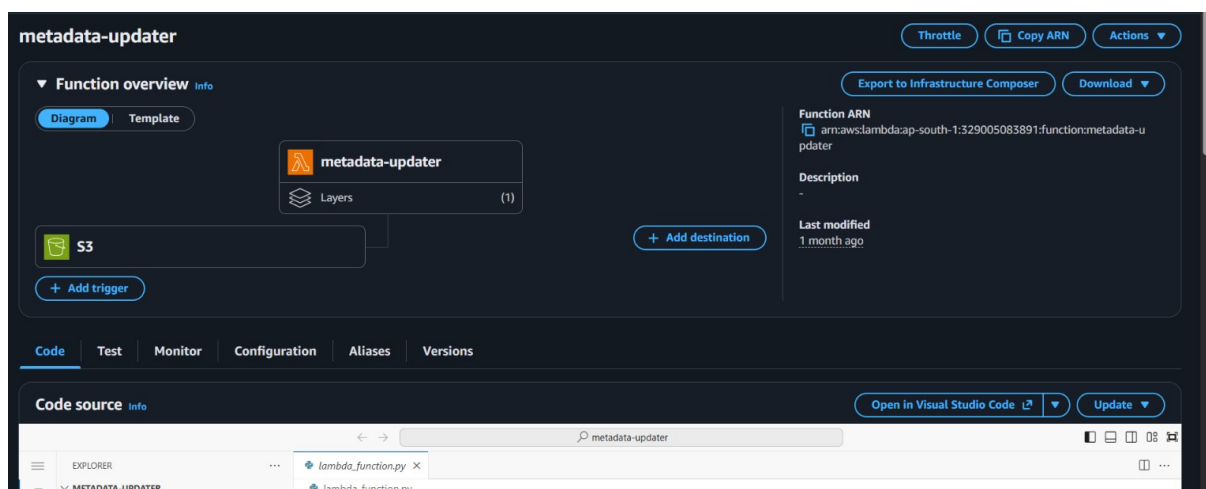
Archive EC2: The Archive EC2 instance provides an isolated storage location for maintaining raw backup copies of the files received from the Analyzer EC2. The Archive subnet is configured without internet access, and the attached storage volume is protected using LUKS encryption, providing secure and isolated backup storage for the DataShield pipeline.

```
[ec2-user@ip-10-60-50-209 ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE  MOUNTPOINTS
nvme1n1      259:0    0   20G  0 disk
└─archive_vault 253:0    0   20G  0 crypt /mnt/archive_vault
nvme0n1      259:1    0    8G  0 disk
├─nvme0n1p1   259:2    0    8G  0 part /
├─nvme0n1p127 259:3    0    1M  0 part
└─nvme0n1p128 259:4    0   10M  0 part /boot/efi

[ec2-user@ip-10-60-50-209 ~]$ ls /mnt/archive_vault
data_20260918161719.txt  lost+found
[ec2-user@ip-10-60-50-209 ~]$
```

## Lambda:

Lambda: An AWS Lambda function was configured to respond to S3 object events generated when processed files are uploaded to the DataShield S3 bucket. The function reads the S3 object information and inserts the corresponding file metadata into the private RDS MySQL database, completing the automated S3-to-RDS metadata update process.



## RDS:

RDS: An Amazon RDS MySQL database was configured in the private DB subnet to securely store DataShield metadata. The database is not publicly accessible and receives metadata updates from Lambda while the Service EC2 securely retrieves the stored records for application responses

**Databases (1)** Group resources Modify Actions Create database

Filter by databases

DB identifier Status Role Engine Upgrade rollout order Region ... Size

mysql-db Stoppe... Instance MySQL Co... SECOND ap-south-1b db.t4g.micro

```
MySQL [metadata]> SELECT * FROM metadata ORDER BY id DESC LIMIT 3;
```

| id | file_name                                    | processed_at        | status    | source            |
|----|----------------------------------------------|---------------------|-----------|-------------------|
| 11 | processed/2026-09-15/data_20260915160252.txt | 2026-09-15 10:40:06 | processed | lambda-s3-trigger |
| 10 | processed/2026-08-12/data_20260811212652.txt | 2026-08-12 06:50:06 | processed | lambda-s3-trigger |
| 9  | processed/2026-08-11/data_20260811133214.txt | 2026-08-11 08:10:04 | processed | lambda-s3-trigger |

3 rows in set (0.002 sec)

```
MySQL [metadata]>
```

## Service ec2:

Service EC2: The Service EC2 instances provide the application and data retrieval layer of the DataShield pipeline. They are deployed through the Auto Scaling Group behind ALB-Project and securely connect to the private RDS MySQL database to retrieve and return the processed metadata records.

/health → Service application is responding successfully.

```
Pretty-print ☐
```

```
{"status": "ok"}
```

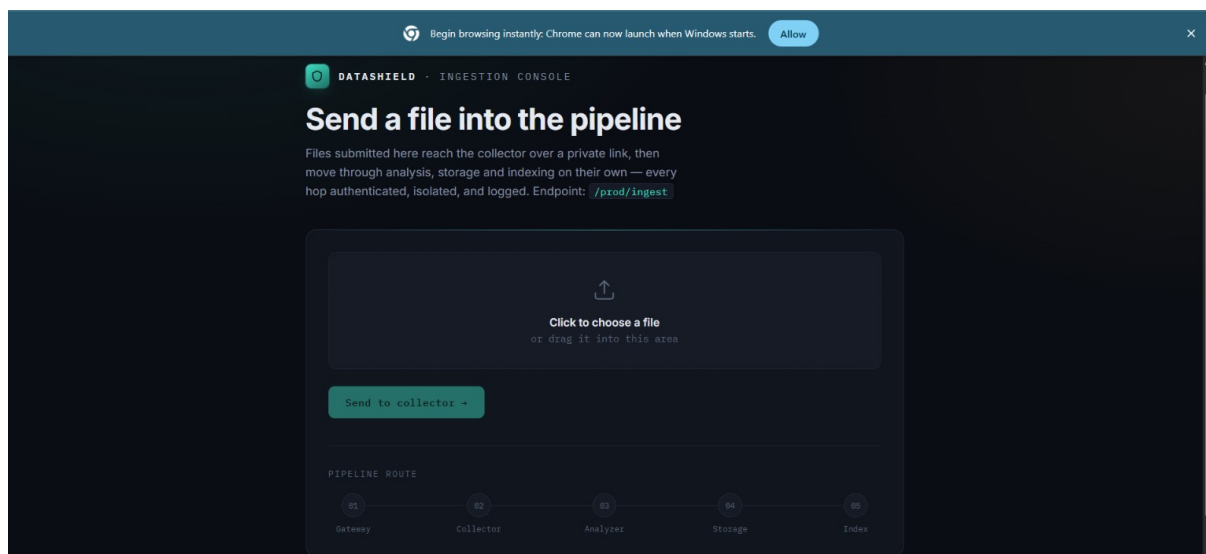
/metadata/latest → Service is successfully retrieving metadata from RDS.

```
Pretty-print ☐
```

```
[["processed/lambda-test.txt", "Wed, 16 Sep 2026 11:34:21 GMT", "processed"], ["processed/2026-09-15/data_20260915160252.txt", "Tue, 15 Sep 2026 10:40:06 GMT", "processed"], ["processed/2026-08-12/data_20260811212652.txt", "Wed, 12 Aug 2026 06:50:06 GMT", "processed"], ["processed/2026-08-11/data_20260811133214.txt", "Tue, 11 Aug 2026 08:10:04 GMT", "processed"], ["processed/2026-08-11/data_20260811033048.txt", "Tue, 11 Aug 2026 04:14:38 GMT", "processed"]]
```

Service EC2 Testing: The Service EC2 backend was successfully tested through the ALB-Project. The /health endpoint returned {"status":"ok"}, confirming that the Service application is running and accessible through the Application Load Balancer. The /metadata/latest endpoint successfully retrieved the latest processed metadata records from the private RDS MySQL database, confirming secure Service-to-RDS connectivity and successful backend operation.

## Frontend page:



## Results / Output

Successfully implemented the DataShield – Segmented Secure Analytics Pipeline with a secure multi-tier AWS network, controlled traffic flow, private database connectivity, automated Linux-based data processing, secure S3 storage, event-driven metadata processing, and scalable Service infrastructure. The implementation establishes separate public, private, database, and isolated archive network segments and demonstrates controlled communication between the different layers.

## Challenges Faced

During the development and implementation of the **DataShield – Segmented Secure Analytics Pipeline**, several technical and operational challenges were encountered. These challenges provided practical experience in Linux administration, AWS cloud services, VPC networking, routing, Security Groups, storage, automation, database connectivity, and scalable infrastructure.

## **1. Designing the VPC and Subnet Architecture**

Creating a single VPC with seven different subnets while maintaining proper separation between public, private, database, and archive resources required careful planning. Each subnet had to be assigned the correct CIDR range and associated with the appropriate route table according to its role.

## **2. Configuring Secure AWS Routing**

Configuring the **Internet Gateway, NAT Gateway, Public Route Table, Private Route Table, Database Route Table, and Archive Route Table** while ensuring that traffic followed the intended paths was one of the major challenges. Particular attention was required to ensure that the Database and Archive subnets did not receive direct internet connectivity.

## **3. Implementing Traffic Isolation Using Security Groups**

Configuring Security Groups for the ALB, Service, Collector, Analyzer, RDS, and Archive tiers required careful planning. Each Security Group had to permit only the traffic required by its respective workload while preventing unauthorized communication between tiers.

## **4. Configuring Private EC2 Communication**

Establishing communication between the Collector, Analyzer, and Service EC2 instances while keeping them inside private subnets required correct routing and Security Group configuration. Ensuring that each component could communicate only with the required tier was an important part of the implementation.

## **5. Automating the Linux Data Pipeline**

Configuring Linux automation for data transfer and processing using **Cron jobs and scripts** required testing to ensure that files were transferred and processed at the required intervals. Log verification was also necessary to confirm successful execution and identify failures.

## 6. Configuring Secure S3 Storage

Configuring Amazon S3 with **versioning, lifecycle management, encryption, and blocked public access** required careful implementation. The S3 configuration also had to support the Analyzer workflow and the event that triggers Lambda.

## 7. IAM Role and Permission Management

Creating and configuring IAM policies and roles for the Analyzer and Lambda workloads required careful permission management. The permissions had to provide the required AWS service access without unnecessarily granting additional privileges.

## 8. Configuring Private RDS Connectivity

Connecting the Service tier and Lambda to the private **RDS MySQL** database while keeping public database access disabled required correct subnet placement, Security Group rules, and database configuration. Ensuring that MySQL traffic used port **3306** only through the authorized path was an important security requirement.

## 9. Integrating Multiple AWS Services

Integrating **EC2, VPC, S3, IAM, RDS MySQL, Lambda, API Gateway, ALB, Target Groups, Launch Template, and Auto Scaling** into a single architecture required careful configuration and troubleshooting. Each service had to work correctly with the network and security boundaries established for the project.

## 10. Configuring High Availability and Scalability

Configuring the **Application Load Balancer, Target Group, Launch Template, and Auto Scaling Group** required correct networking, target registration, health checks, and scaling configuration. Maintaining the required Service capacity between **2 and 6 instances** while allowing the ALB to route traffic to healthy instances was an important part of the implementation.

## Overall Challenge Summary

Although the project involved several technical challenges, addressing these issues through planning, configuration, testing, and troubleshooting provided

valuable practical experience. Completing the DataShield project improved practical understanding of **Linux Administration, AWS Cloud Infrastructure, VPC Networking, Routing, Security Groups, IAM, Automation, S3, RDS, Lambda, Load Balancing, and Auto Scaling**, while demonstrating how secure and isolated cloud infrastructure can be designed and implemented.

## Conclusion

The **DataShield – Segmented Secure Analytics Pipeline** successfully demonstrates the design and implementation of a secure, automated, and scalable multi-tier analytics platform using **Linux and Amazon Web Services (AWS)**. The project integrates Linux administration and automation with AWS networking and cloud services to create a controlled workflow for data ingestion, collection, analysis, processing, storage, metadata management, and service delivery.

Throughout the project, AWS services including **VPC, EC2, S3, IAM, RDS MySQL, Lambda, API Gateway, Application Load Balancer, Target Groups, Launch Template, and Auto Scaling** were configured according to the defined architecture. Linux-based automation and scheduled processing were used to support the movement and processing of data between the Collector and Analyzer tiers.

The project places strong emphasis on **network segmentation and traffic isolation**. The VPC uses separate public, private, database, and isolated archive subnets. The Internet Gateway provides connectivity for the public edge, while the NAT Gateway provides controlled outbound connectivity for selected private workloads. The Database and Archive subnets use local-only routing, preventing direct internet access.

Security was further strengthened through dedicated Security Groups and IAM roles. The Service tier can communicate with the private RDS database through MySQL port 3306, while unauthorized traffic is restricted. The Archive tier remains isolated from the internet, and S3 public access is restricted while versioning and lifecycle management are enabled.

The project also demonstrates automation and scalability through Linux Cron jobs, S3 event-driven Lambda processing, Application Load Balancing, and Auto Scaling. The Service tier is designed to maintain a minimum of **2 instances** and scale up to **6 instances** according to CPU utilization.

During implementation, challenges involving VPC design, routing, Security Groups, Linux automation, IAM permissions, private RDS connectivity, AWS service integration, and scalability were addressed through configuration, testing, and troubleshooting. These experiences provided practical understanding of how different AWS and Linux components work together in a secure cloud environment.

Overall, **DataShield achieved its objective of building a segmented and secure analytics platform focused on VPC networking, traffic flow, isolation, automation, and scalability.** The project provided valuable hands-on experience in **AWS Cloud, Linux Administration, Networking, Security, Automation, Storage, Database Connectivity, Load Balancing, and Cloud Infrastructure Management**, creating a strong foundation for professional roles in Cloud Computing, AWS Administration, Linux System Administration, DevOps, and Infrastructure Management.